

Przegląd drzewa bez dodatkowej pamięci

(bez rekurencji, stosu, dodatkowych bitów w węźle itp.)

Oznaczmy:

- $w = v\$$ jeśli w jest następnym po v węzłem w porządku inorder,
- $w = \$v$ jeśli poprzednim.

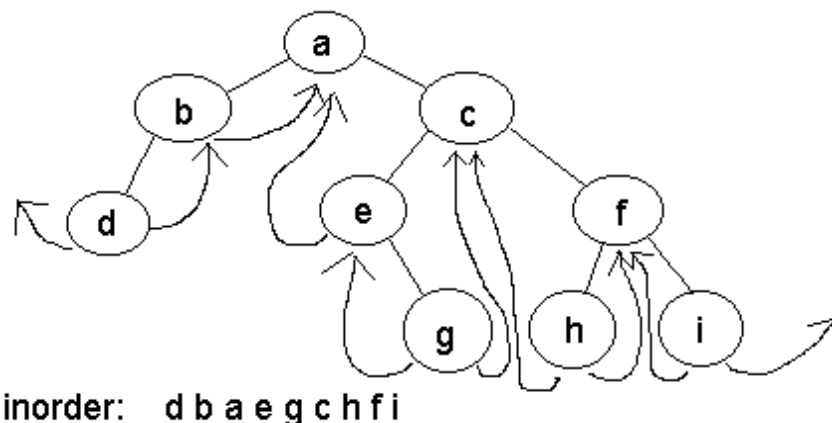
Drzewo binarne nizane (przeszyte, ang. threaded):

- Jeśli $\text{right}(v) = \text{NULL}$, to zamiast NULL wstawiamy "przeszycie" $\text{right}(v) = v\$$;
- dodatkowo w węźle v ustawiamy flagę określającą znaczenie linku;
- analogicznie dla poprzednika: $\text{left}(v) = \$v$.

Aby otrzymać postać nizaną drzewa trzeba go oczywiście przeglądnąć w porządku inorder.

Mając już ustawione przeszycia:

- możemy łatwo wykonywać operację $v := v\$$, czyli przeglądać drzewo, bez rekurencji, bez stosu,
- przegląd możemy zacząć od dowolnie wskazanego wierzchołka drzewa.



Koszt pamięciowy to flagi: 1 bit w każdym węźle.

Algorytm Morrisa

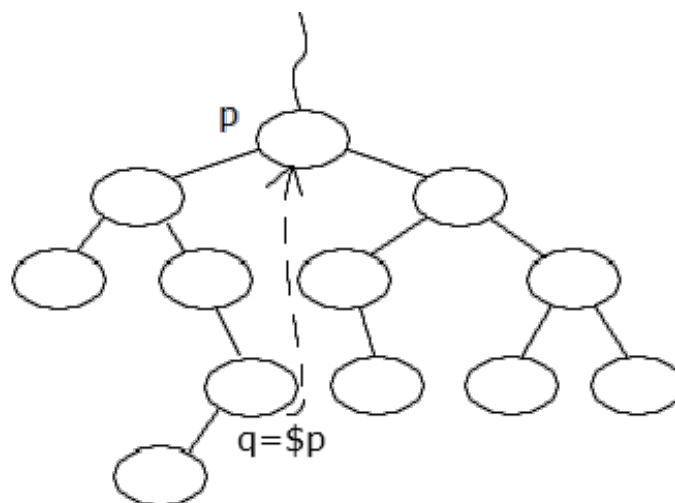
- preorder/inorder, bez stosu/rekurencji, bez flag itp.
- wykorzystuje ideę drzew niszczonych: na chwilę wstawia w puste pole prawego wskaźnika odsyłacz do następnego węzła w kolejności inorder.

Pomysł:

- jestem w węźle p;
- przed wejściem w lewe poddrzewo, aby móc wrócić do p i pójść w prawo, zapamiętuję wskaźnik do p w ostatnim (w inorder) węźle q lewego poddrzewa;
- aby odszukać węzeł q idę raz w lewo a potem do końca w prawo (cały czas aktualna pozycja to p);
- po wejściu w lewe poddrzewo i zbadaniu węzła q tym linkiem wrócę do p.

Jak stwierdzić, że jestem w węźle p po raz drugi?

- stojąc w węźle p wykonujemy trawers raz w lewo i do końca w prawo;
- jeśli natrafimy na NULL to znaczy że w węźle p jesteśmy po raz pierwszy;
- jeśli natrafimy na p to znaczy że jesteśmy w węźle p po raz drugi, musimy więc odtworzyć NULL w węźle q=\$p i przejść do prawego poddrzewa.



```

p := root;
while (p ≠ NULL) do
    if (left(p) = NULL)          // przyszliśmy z góry
        PreorderVisit(p); InorderVisit(p);
        p := right(p);
    else // sprawdź czy przyszliśmy do węzła p z góry,
        // czy z dołu, przeszyciem
        q := left(p);
        while (right(q) ≠ NULL AND right(q) ≠ p)
        (*)    q := right(q);
        if (right(q) = NULL) then
            // p = q$, w p jesteśmy po raz pierwszy
            PreorderVisit(p);
            right(q) := p; // wstawiliśmy przeszycie
            p := left(p);
        else // right(q)=p, przeszycie
            // którym przeszliśmy przed chwilą z q do p
            right(q) := NULL; //odtworzyliśmy oryg. link
            InorderVisit(p);
            p := right(p);
end.

```

Ile razy wykonuje się iteracja (*) ?

- rozważmy dowolną krawędź na prawym grzbiecie lewego poddrzewa węzła p;
- przejście tą krawędzią w pętli (*) wykonuje się tylko wtedy, gdy aktualne położenie jest w p;
- a zatem dwukrotnie w całym algorytmie;
- a więc (*) wykonuje się co najwyżej 2 razy liczba prawych krawędzi.

Pozostałych operacji jest oczywiście $\Theta(n)$.

Zatem algorytm Morrisa ma złożoność liniową.

Nie znamy efektywnej wersji dla przeglądu postorder.

Algorytm "potrójnego porządku"

(Lindstrom, Dwyer)

Pomysł: rotacja odsyłaczy (w kółko), dla pól:

q – poprzednik w drzewie (rodzic),
p – aktualny węzeł,
left(p),
right(p).

```
p := root;  // aktualny węzeł
q := head;
    // poprzednik na trasie przeglądu, początkowo
    // nagłówek drzewa lub dowolny niepusty adres,
    // byleby nie wskazywał na żaden węzeł drzewa

while (p ≠ head) do
    if (p = NULL) then          // wróć do poprzednika
        p := q;
        q := NULL;
    else
        zbadaj(p);
        // idziemy dalej ZAWSZE wzdłuż p->left,
        // ale wychodząc rotujemy linki
        tmp      := left(p);
        left(p)  := right(p);
        right(p) := q;
        q        := p;
        p        := tmp;
end.
```

- p wskazuje na aktualny węzeł,
- q na węzeł z którego przyszliśmy,
- do q wrócimy po eksploracji poddrzewa o korzeniu p.

Rotacja - czyli kolejny krok:

- w pole `left(p)` wprowadza dotychczasową wartość wskaźnika `right(p)`,
- w pole `right(p)` dotychczasową wartość `q` czyli poprzednika,
- `q` przesuwa na aktualne położenie, czyli `p`;
- `p` przesuwa krok dalej, czyli zgodnie z dotychczasową wartością `left(p)`.

Po powrocie do węzła `p` z lewego poddrzewa znowu pójdziemy wzdłuż `left(p)`, ale:

- teraz ten adres wskaże prawe poddrzewo,
- a rotacja wskaźników spowoduje, że `left(p)` stanie się adresem poprzednika w drzewie (link ten ze zmiennej `q` przejdzie przez `right(p)` do `left(p)`).

Zatem po powrocie z prawego poddrzewa pójdziemy, zgodnie z aktualną wartością `left(p)`, do poprzednika, czyli w górę drzewa.

Rezultat:

- każdy węzeł badany 3 razy, ale
- nie można określić kiedy w którym z trzech porządków.

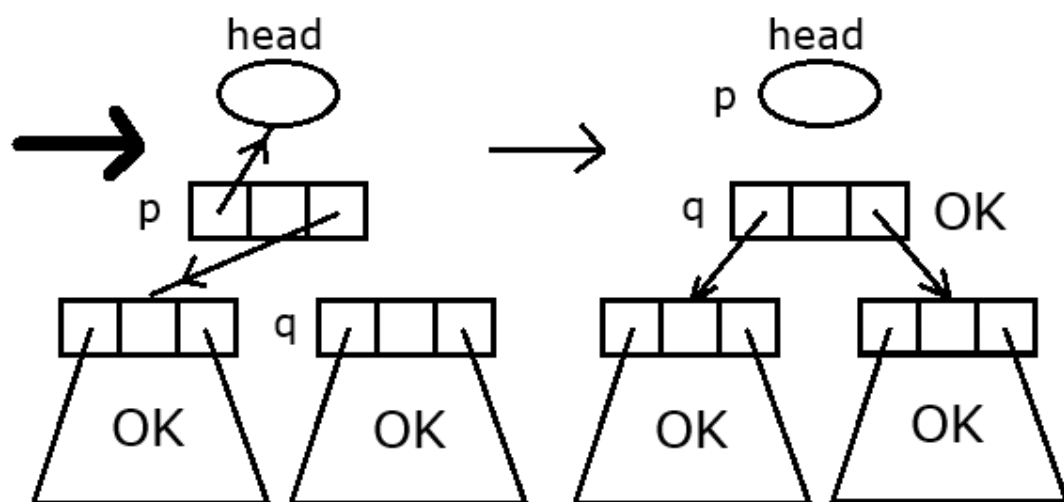
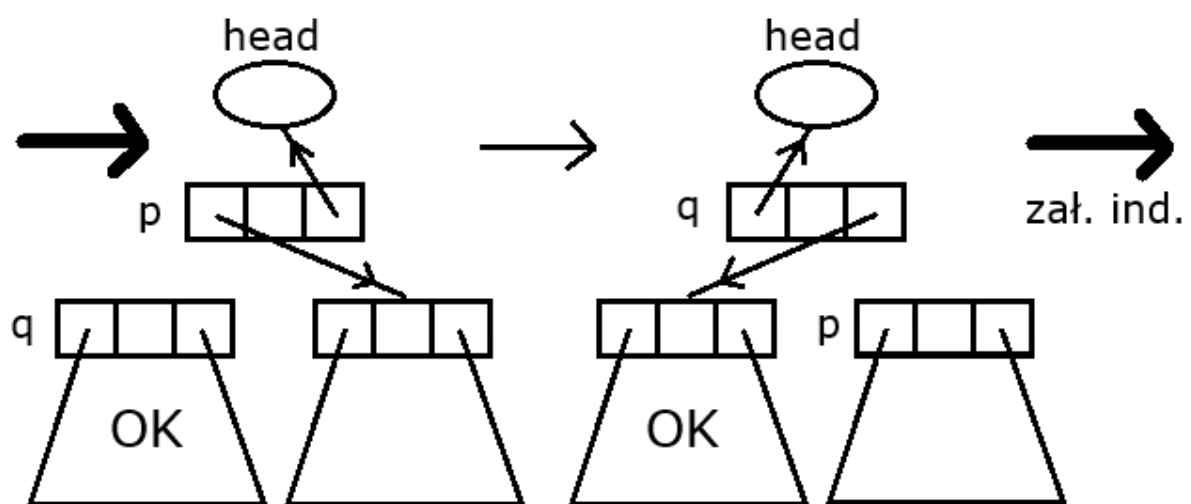
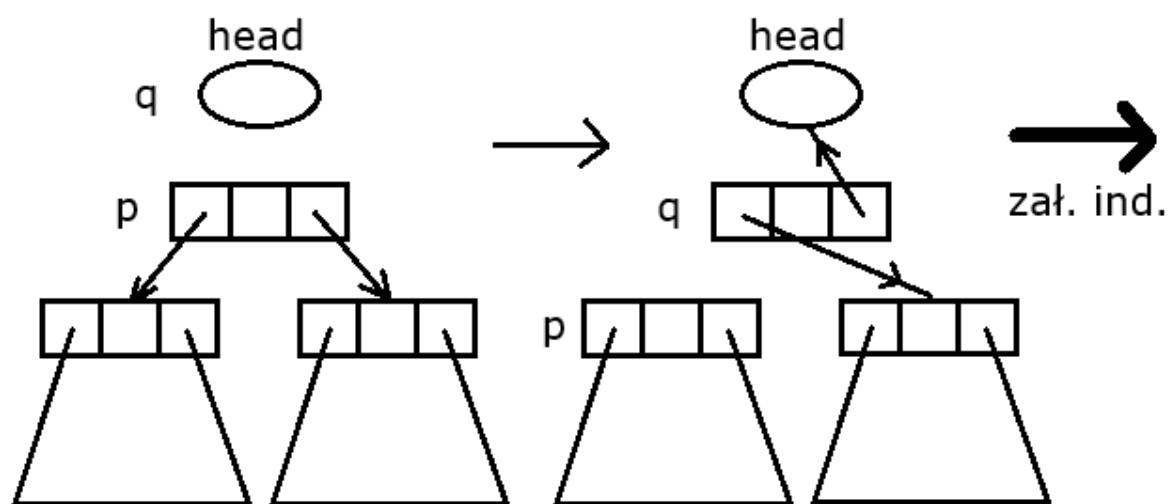
Poprawność: indukcją na budowę drzewa!

Wykazujemy, że po wykonaniu algorytmu:

- `p=head`,
- `q=root`,
- wszystkie pola wskaźnikowe w drzewie mają wartość oryginalną.

drzewo puste: 1 iteracja pętli `while`, OK.

krok indukcyjny: 6 obrazków :)



Algorytm Robsona (materiał nieobowiązkowy)

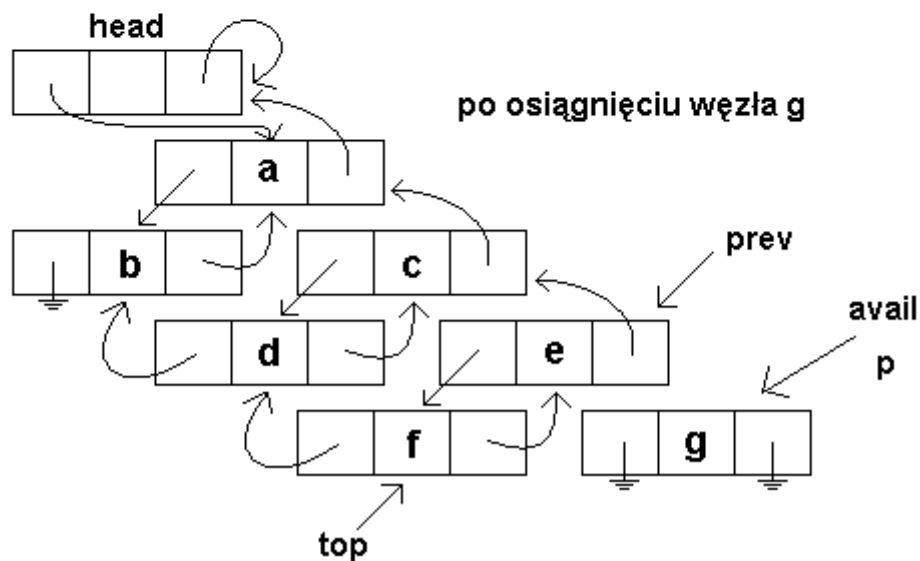
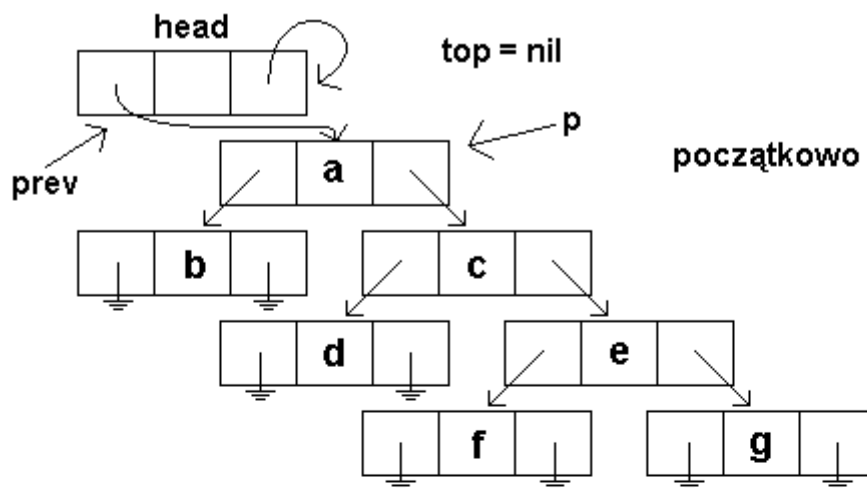
Pełne rozwiązanie problemu przeglądu drzewa binarnego:

- bez rekurencji
- bez stosu
- bez dodatkowych bitów
- w każdym (dowolnym) z trzech porządków

Metoda:

- odwracając odsyłacze:
 - gdy przesuwamy się do lewego poddrzewa to odwracamy odsyłacz left; gdy do prawego – odwracamy right.
 - w ten sposób tworzymy ścieżkę powrotną (odpowiadającą stosowi rekurencji w algorytmie klasycznym) za pomocą odwróconych odsyłaczy.
- powstaje problem podczas powrotu: jak rozpoznać z którego poddrzewa wracamy – czyli który odsyłacz trzeba zrekonstruować?
- jeśli tylko jedno poddrzewo niepuste w danym węźle, to problemu nie ma;
- jeśli oba niepuste, i jeśli idziemy w prawe poddrzewo, to zapamiętujemy adres węzła na pomocniczym stosie;
- stos pomocniczy organizowany jest w liściach, za pomocą pól odsyłaczy do poddrzew – oba te pola w liściu są puste; w liściu leżącym na stosie prawy odsyłacz pamięta adres węzła w którym nastąpiło przejście w prawo, lewy odsyłacz wskazuje poprzedni węzeł na stosie;

- liście które już odwiedziliśmy wystarczają do utworzenia stosu pomocniczego: mianowicie z każdym węzłem stopnia 2 można skojarzyć unikalny liść leżący w jego lewym poddrzewie, jest to ostatni napotkany liść, zapamiętywany w zmiennej avail;
- gdy wracamy z poddrzewa do węzła stopnia 2, to jego adres jest na szczycie stosu wtedy i tylko wtedy gdy wracamy z prawego poddrzewa; na rys. poniżej liście b, d, f tworzą ten pomocniczy stos.



Algorytm Robson:

```
prev := head;           // drzewo z nagłówkiem,
left(head) := root
avail := NULL;          // wskaźnik do wolnej komórki
top := NULL;
p := left(head);
dir := left_or_right;

while TRUE do

    if dir = left_or_right then
        PreorderVisit(p);
        if left(p) ≠ NULL then
            // w lewo, odwracając lewy odsyłacz do prev
            q := left(p);
            left(p) := prev;
            prev := p;
            p := q;

        else if right(p) ≠ NULL then
            InorderVisit(p);
            // w prawo odwracając prawy odsyłacz do prev
            q := right(p);
            right(p) := prev;
            prev := p;
            p := q;

        else // p jest liściem
            InorderVisit(p);
            PostorderVisit(p);
            avail := p;
            dir := back;

    else // dir = back

        if prev = head then
            left(head) := p;
            return;

        if left(prev) = NULL then
            // powrót z prawej, lewego następnika nie było,
            // wystarczy odwrócić odsyłacz
```

```

    q := right(prev);
    right(prev) := p;
    p := prev;
    prev := q;
    PostorderVisit(p);

else if right(prev) = NULL then
    // powrót z lewej, bez przejścia w prawo
    q := left(prev);
    left(prev) := p;
    p := prev;
    prev := q;
    InorderVisit(p);
    PostorderVisit(p)

else // oba poddrzewa węzła prev niepuste
    if top = NULL or right(top) ≠ prev then
        // powrót z lewej, idź w prawo,
        // zapamiętaj węzeł na stosie
        q := left(prev);
        left(prev) := p; //lewy wsk. przywrócony
        p := right(prev);
        right(prev) := q; // prawy wsk. odwrócony
        // teraz operacja push
        right(avail) := prev;
        left(avail) := top;
        top := avail;
        dir := left_or_right;
        InorderVisit(prev);

    else // powrót z prawej
        PostorderVisit(prev);
        q := right(prev);
        right(prev) := p; //prawy wsk. przywrócony
        p := prev;
        prev := q;
        stack := left(top);
        left(top) := right(top) := NULL;
        //odsyłacze w liściu OK
        top := stack; // = pop
end while.

end Robson.

```